Lab - 10 : Implementation of Distance Vector Routing Algorithm in C++

ID - Card

Name: Srimoneyshankar Ajith
Roll_Number : 2024BCS0055

Aim

To design and implement the Distance Vector Routing (DVR) algorithm using C++ to simulate the exchange of distance vectors, update routing tables using the Bellman-Ford equation, and determine the optimal shortest paths between routers in a network topology.

Requirements

Hardware

- 1 PC or Laptop

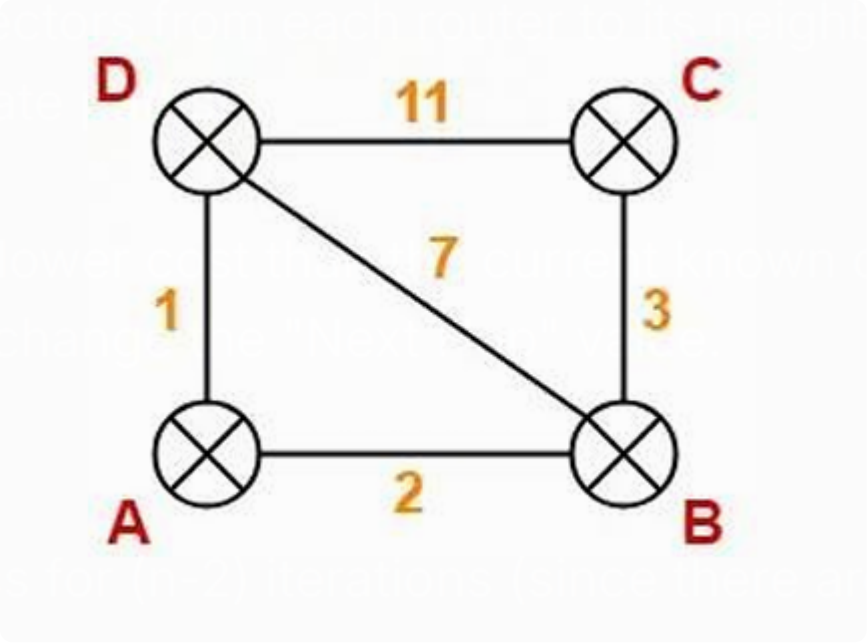
Software

- C++ Compiler (GCC, Clang, or MSVC)
- IDE or Text Editor (VS Code, etc.)
- Language : C++ (Standard Template Library)

Procedure

1. Network Topology Definition
   Open the C++ IDE.
   Create a new file named.
   Define a network consisting of 4 routers (A, B, C, D) using an adjacency matrix based on the given network costs:

Cost(A → B) = 2
Cost(A → D) = 1
Cost(B → C) = 3
Cost(B → D) = 7
Cost(C → D) = 11



2. Initialize Routing Tables (Step-01)
   Create a 2D array for the distance table and another for the "next hop" table.
   Populate each router's initial routing table using its local knowledge (direct neighbors).
   Assign INF (Infinity) to routes that are not directly connected.
   Print the initial routing tables for all routers.

3. Exchange Distance Vectors (Step-02 & Step-03)
   Simulate the broadcasting of distance vectors from each router to its neighboring routers.
   Apply the Bellman-Ford equation to update paths:
   Dx(y) = min{C(x,v) + Dv(y)}
   If a route through a neighbor provides a lower cost than the current known distance, update the routing table with the new distance and change the "Next Hop" value.

4. Execute Iterations
   Repeat the exchange and update process for (n-2) iterations (since there are 4 routers, loop 2 times to ensure convergence).
   Print the state of all routing tables at each step to verify the dynamic updates.

5. C++ Source Code

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

const int inf = 1e9;
const int num_routers = 4;

void print_routing_tables(int step, int dist_table[][num_routers], int
next_hop[][num_routers]) {
    cout << "Step-0" << step << ":" << endl;
    for (int i = 0; i < num_routers; i++) {
        cout << "Routing Table at Router " << char('A' + i) << endl;
        cout << "Destination\tDistance\tNext Hop" << endl;
        for (int j = 0; j < num_routers; j++) {
            cout << char('A' + j) << "\t\t";
            if (dist_table[i][j] == inf) cout << "INF\t\t";
            else cout << dist_table[i][j] << "\t\t";

            if (next_hop[i][j] != -1) cout << char('A' + next_hop[i][j]);
            else cout << "-\t";
            cout << endl;
        }
        cout << endl;
    }
    cout << "--------------------------------------------------" << endl;
}

int main() {
    int cost_matrix[num_routers][num_routers] = {
        {0,   2,   inf, 1},
        {2,   0,   3,   7},
        {inf, 3,   0,   11},
        {1,   7,   11,  0}
    };

    int dist_table[num_routers][num_routers];
    int next_hop[num_routers][num_routers];

    for (int i = 0; i < num_routers; i++) {
        for (int j = 0; j < num_routers; j++) {
            dist_table[i][j] = cost_matrix[i][j];
            if (i == j) next_hop[i][j] = i;
            else if (cost_matrix[i][j] != inf) next_hop[i][j] = j;
            else next_hop[i][j] = -1;
```

```
        }
    }

    print_routing_tables(1, dist_table, next_hop);

    for (int step = 2; step ≤ 3; step++) {
        int next_dist_table[num_routers][num_routers];
        int next_next_hop[num_routers][num_routers];

        for (int i = 0; i < num_routers; i++) {
            for (int j = 0; j < num_routers; j++) {
                next_dist_table[i][j] = dist_table[i][j];
                next_next_hop[i][j] = next_hop[i][j];
            }
        }

        for (int i = 0; i < num_routers; i++) {
            for (int j = 0; j < num_routers; j++) {
                for (int k = 0; k < num_routers; k++) {
                    if (i ≠ k && cost_matrix[i][k] ≠ inf) {
                        if (dist_table[k][j] ≠ inf) {
                            int new_dist = cost_matrix[i][k] + dist_table[k]
[j];
                            if (new_dist < next_dist_table[i][j]) {
                                next_dist_table[i][j] = new_dist;
                                next_next_hop[i][j] = k;
                            }
                        }
                    }
                }
            }
        }

        for (int i = 0; i < num_routers; i++) {
            for (int j = 0; j < num_routers; j++) {
                dist_table[i][j] = next_dist_table[i][j];
                next_hop[i][j] = next_next_hop[i][j];
            }
        }
        print_routing_tables(step, dist_table, next_hop);
    }
    return 0;
}
```

Output Screenshots

1. Step-01 Output

```
 sms on Aether via  v13.3.0 at …/4th Semester/ICS224/SMS ❯ ./a.out
Step-01:
Routing Table at Router A
Destination     Distance        Next Hop
A               0               A
B               2               B
C               INF             -
D               1               D

Routing Table at Router B
Destination     Distance        Next Hop
A               2               A
B               0               B
C               3               C
D               7               D

Routing Table at Router C
Destination     Distance        Next Hop
A               INF             -
B               3               B
C               0               C
D               11              D

Routing Table at Router D
Destination     Distance        Next Hop
A               1               A
B               7               B
C               11              C
D               0               D
```

2. Step-02 Output

```
-----------------------------------------------------
Step-02:
Routing Table at Router A
Destination     Distance        Next Hop
A               0               A
B               2               B
C               5               B
D               1               D

Routing Table at Router B
Destination     Distance        Next Hop
A               2               A
B               0               B
C               3               C
D               3               A

Routing Table at Router C
Destination     Distance        Next Hop
A               5               B
B               3               B
C               0               C
D               10              B

Routing Table at Router D
Destination     Distance        Next Hop
A               1               A
B               3               A
C               10              B
D               0               D
```

3. Step-03 Output (Converged State)

```
-------------------------------------------------
Step-03:
Routing Table at Router A
Destination     Distance        Next Hop
A               0               A
B               2               B
C               5               B
D               1               D

Routing Table at Router B
Destination     Distance        Next Hop
A               2               A
B               0               B
C               3               C
D               3               A

Routing Table at Router C
Destination     Distance        Next Hop
A               5               B
B               3               B
C               0               C
D               6               B

Routing Table at Router D
Destination     Distance        Next Hop
A               1               A
B               3               A
C               6               A
D               0               D
```

Result

The C++ program successfully simulated the Distance Vector Routing algorithm. The routing tables for routers A, B, C, and D were initialized and iteratively updated using distance vectors from neighboring nodes. The algorithm converged correctly, yielding the optimal shortest paths and next-hop information for the entire network.